

Spark Structured Streaming

What is streaming?

- Streaming means processing data continuously as it arrives, instead of waiting until all the data has been collected
- Process data continuously as it arrives

What is structured streaming?

- Spark structure streaming is the stream-processing engine in Apache Spark that allows you to process data continuously as it arrives, using the same DataFrame and spark SQL APIs you already use for batch processing
- Structured streaming = spark DataFrame + continuous incoming data
- **Spark Batch**: read → transform → write
- **Spark structure streaming**: readStream → transform → writeStream
- Example

```
df = (
    spark.readStream
      .format("json")
      .schema(schema)
      .load("/data/orders")
)

result = df.filter(df.amount > 100)

query = (
    result.writeStream
      .format("console")
      .outputMode("append")
      .start()
)
```

What is the different between batch and streaming?

- **Batch**: process data that already exists
- **Streaming**: process data continuously as it arrives

Stateless Transformations

- Is a transformation where spark processes each incoming record without needing to remember information from previous records
- Examples
 - o select(), fileter(), withColumn(), drop(), cast(), union(), map()

Stateful Transformations

- is a transformation where spark must remember information from previous data

- example
 - o `groupBy().sum()`, `groupBy().count()`, `groupBy().agg()`, window aggregation

Checkpoint Director

- checkpoint is a location where spark structured streaming stores information about the progress and state of a streaming query
- checkpoint = spark's recovery/storage mechanism for a streaming query
- it contains information such as
 - o **commits:**
 - information about successfully committed micro-batches
 - which batches finished
 - o **offsets:**
 - offset/progress information for the source
 - how far have I read?
 - o **Source:**
 - Information about the streaming sources
 - Where am I reading data from?
 - o **State:**
 - Stateful operator state
 - What previous state do I need to remember?
- example

```
query = (
  df.writeStream
    .format("parquet")
    .option("path", "/data/output")
    .option("checkpointLocation", "/data/checkpoint")
    .start()
)
```

Output Modes

- output mode tells spark which rows/results should be written to the sink every time the streaming query producers an updated result
- the easiest way to understand them is with a streaming aggregation
- there are 3 main output modes
 - 1- append mode**
 - only new rows that are finalized are written to the output
 - spark doesn't rewrite old output rows
 - usually useful for stateless transformations
 - 2- complete mode**
 - write the entire current result table every time the result changes
 - mainly useful for stateful aggregation

3- update mode

- only rows whose results changed since the previous trigger are written

Trigger & Triggers Types

- trigger determine when spark should process the available data
- think of it: when should spark execute another micro-batch?
- Types of triggers
 - **Default:** process continuously as fast as possible s
 - **processingTime:** process at a fixed time interval
 - **availableNow:** process all currently available data, then stop
 - **Once:** process one batch, then stop
 - **Continuous:** low-latency continuous processing mode
- Example

```
query = (  
    df.writeStream  
        .format("console")  
        .trigger(processingTime="10 seconds")  
        .start()  
)
```

- Example

```
query = (  
    df.writeStream  
        .format("parquet")  
        .trigger(availableNow=True)  
        .start()  
)
```

Trigger	Keeps running?	Processing style	Main use
Default	✓	As fast as possible	Normal streaming
ProcessingTime	✓	Micro-batch at interval	Scheduled intervals
AvailableNow	✗	Process all available data	Incremental ETL
Once	✗	One micro-batch	One-time processing
Continuous	✓	Continuous low-latency	Specialized use cases

foreachBatch()

- It allows you to run custom code on each micro-batch produced by a spark structured streaming query
- Take every streaming micro-batch process it using normal spark DataFrame operations
- means every time spark creates a micro-batch, run my function on that batch. That is
- Syntax

```
def process_batch(batch_df, batch_id):  
    # normal DataFrame operations  
    batch_df.show()
```

```

query = (
    df.writeStream
      .foreachBatch(process_batch)
      .start()
)

query.awaitTermination()

```

Event Time vs Processing Time

- **Event Time:**
 - when the event actually happened
 - is the timestamp attached to the event by the producer/source
 - when the data happened
- **Processing Time:**
 - when spark processes the event
 - is the time when spark actually processes the event

```

result = (
    df
      .groupBy(
        window("event_time", "5 minutes")
      )
      .agg(count("*").alias("orders"))
)

```

Window Operations in Streaming

- windowing is a way to divide a continuous stream of data into time-based groups (windows) so that you can perform calculations on each group
- window = group streaming events according to time, then perform an operation on each time period
- types of windowing
 - tumbling window
 - sliding window
 - session window

Tumbling Window Operation

- tumbling window has
 - fixed duration
 - no overlap
 - each event belongs to one window only
- example

```

result = (
    df
      .groupBy(
        window("event_time", "5 minutes")
      )
      .agg(
        count("*").alias("order_count")
      )
)

```

```
)  
)
```

Sliding Window Operation

- sliding window has
 - o fixed window duration
 - o sliding interval
 - o windows can overlap
 - o an event can belong to multiple windows
- examples

```
result = (  
    df  
    .groupBy(  
        window(  
            "event_time",  
            "10 minutes",  
            "5 minutes"  
        )  
    )  
    .agg(  
        count("*").alias("order_count")  
    )  
)
```

Session Window Operation

- session window groups events that belong to the same activity/session of a user, where the session ends after the user has been inactive for a certain amount of time
- there is a period of inactivity called the gap
- group events based on activity or inactivity
- example

```
result = df.groupBy(  
    "user_id",  
    session_window("timestamp", "10 minutes")  
) .count()
```

Handle Late Arrival Events

- late event means an event arrives at the streaming system after the time when it actually happened, suppose an event actually happened at **10:02** but because of network delay spark receives it at **10:08**
- the main tool for handling late events is a watermark
 - o watermark tells spark how long it should wait for events that arrive late according to their event time

```
result = (  
    df  
    .withWatermark("event_time", "10 minutes")  
    .groupBy(  
        window("event_time", "5 minutes")  
    )  
    .count()  
)
```

Watermarks for Structured Streaming

- a watermark is a mechanism in spark structured streaming that helps spark handle late-arriving events and cleanup old state
- **watermark** = a limit on how late event-time data can be for a stateful streaming operations
- means spark can use the watermark to determine when old event-time data is too late to keep affecting state and when old can be cleaned up

```
result = (
  df
    .withWatermark("event_time", "10 minutes")
    .groupBy(
      window("event_time", "5 minutes")
    )
    .agg(
      count("*").alias("event_count")
    )
)
```

- **Event Time:** when did the event actually happen?
 - **Window:** which time period does the event belong to?
 - **Watermark:** how far has spark progressed in event time, and how much lateness will it tolerate for state management
 - **Late Event:** an event whose time behind the stream's current event-time progress
- 🔧 A **watermark** helps spark structured streaming handle late events and remove old state so it doesn't grow forever